

9. Header Linked List

–Delete At Position

Program:

```
void deleteFromPosition(int pos)
{
    if (head->next == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    if (pos < 1 || pos > head->data)
    {
        cout << "Invalid position!" << endl;
        return;
    }

    Node *prev = head;

    // Traverse to the node BEFORE the target position
    for (int i = 1; i < pos; i++)
    {
        prev = prev->next;
    }

    Node *nodeToDelete = prev->next;

    // Unlink the node
    prev->next = nodeToDelete->next;

    cout << "Deleted " << nodeToDelete->data << " from
position " << pos << "." << endl;

    free(nodeToDelete);
    nodeToDelete = nullptr;

    // Decrement internal count
```

```

    head->data--;
}
.....
case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

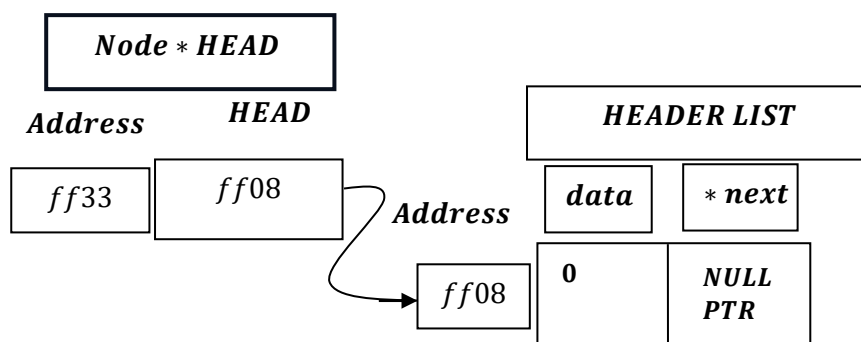
Program Analysis

```

if (head->next == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

```

i. e.



head → next = NULLPTR , head → Data = 0 , then list is empty .

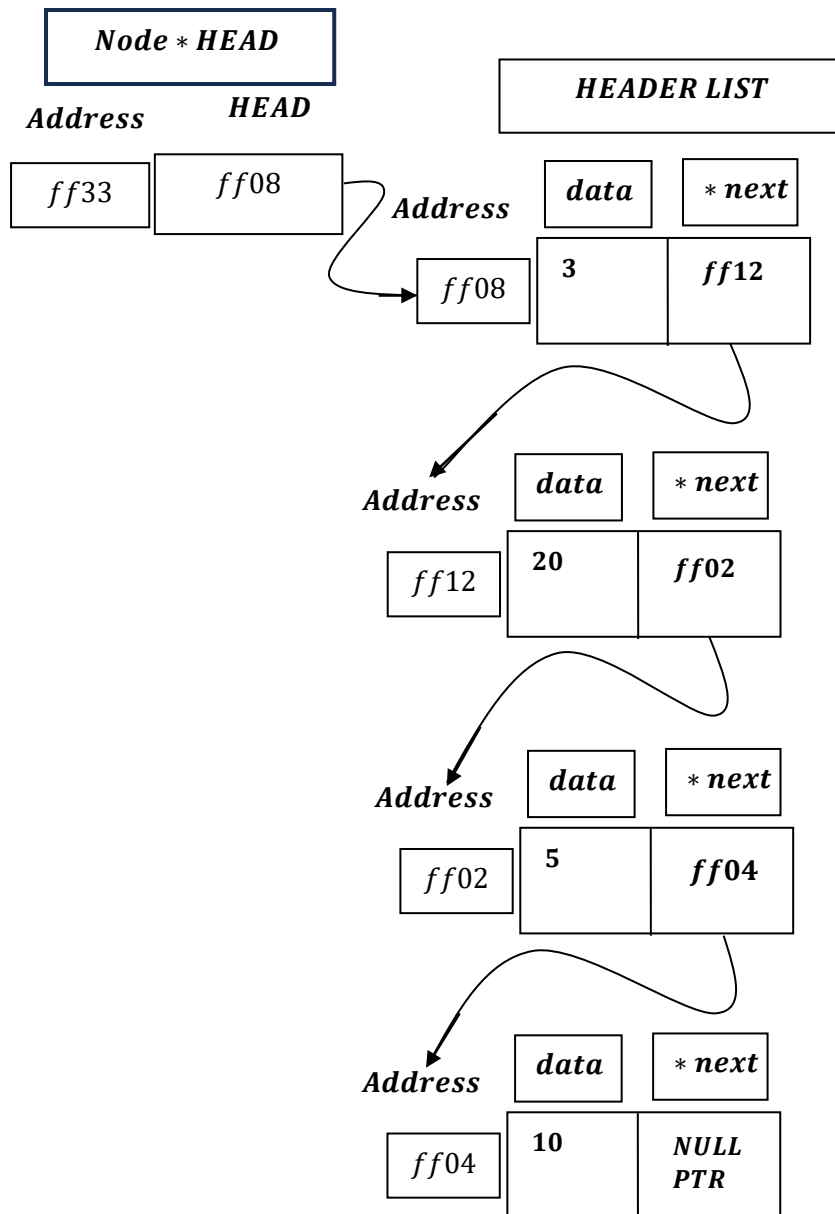
```
if (pos < 1 || pos > head->data)
{
    cout << "Invalid position!" << endl;
    return;
}
```

Now, if list has 4 nodes, Header Node's Data is also 4.

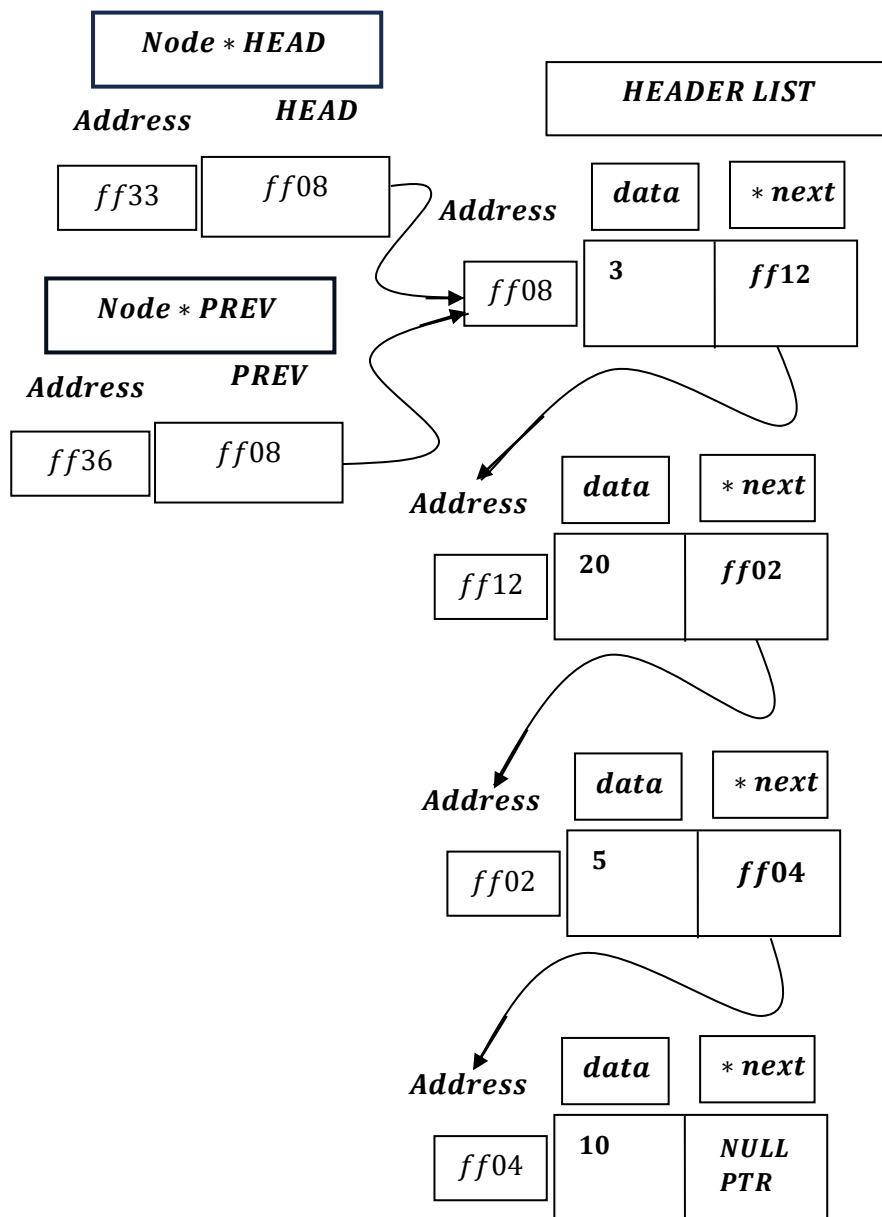
Now we can delete header node's data at position: 1, 2, 3, 4. Greater than that of four (4) will result: Invalid Position.

Therefore, when `INVALID POSITION` : When $POS < 1$ or $Pos > \text{Header Node's Data}$.

If List is not Empty



```
Node *prev = head;  
Node * Prev = head = ff08.
```

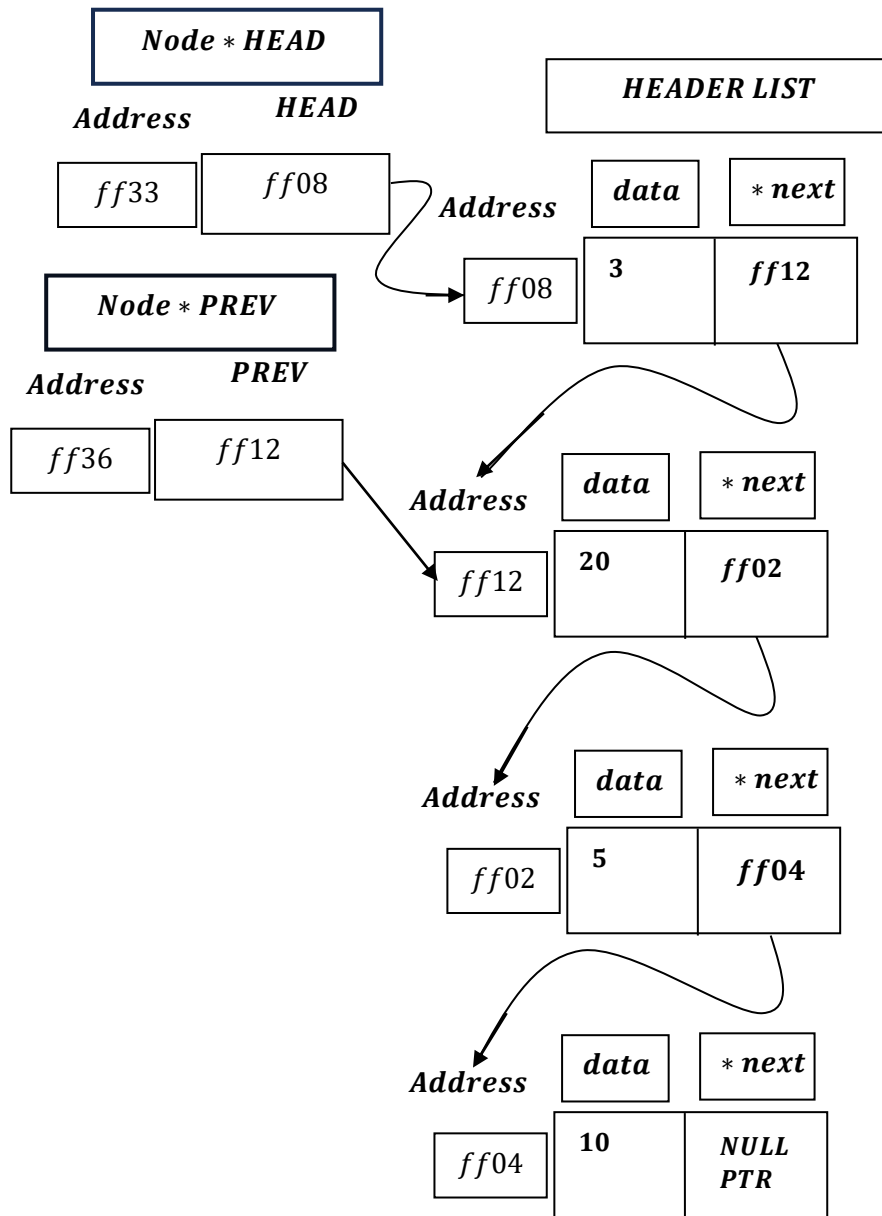


```
for (int i = 1; i < pos; i++)
{
    prev = prev->next;
}
```

Say, $Pos = 2$.

$int\ i = 1 ; i = 1 < Pos: 2\ [True]:$

$PREV = PREV \rightarrow NEXT = ff12;$

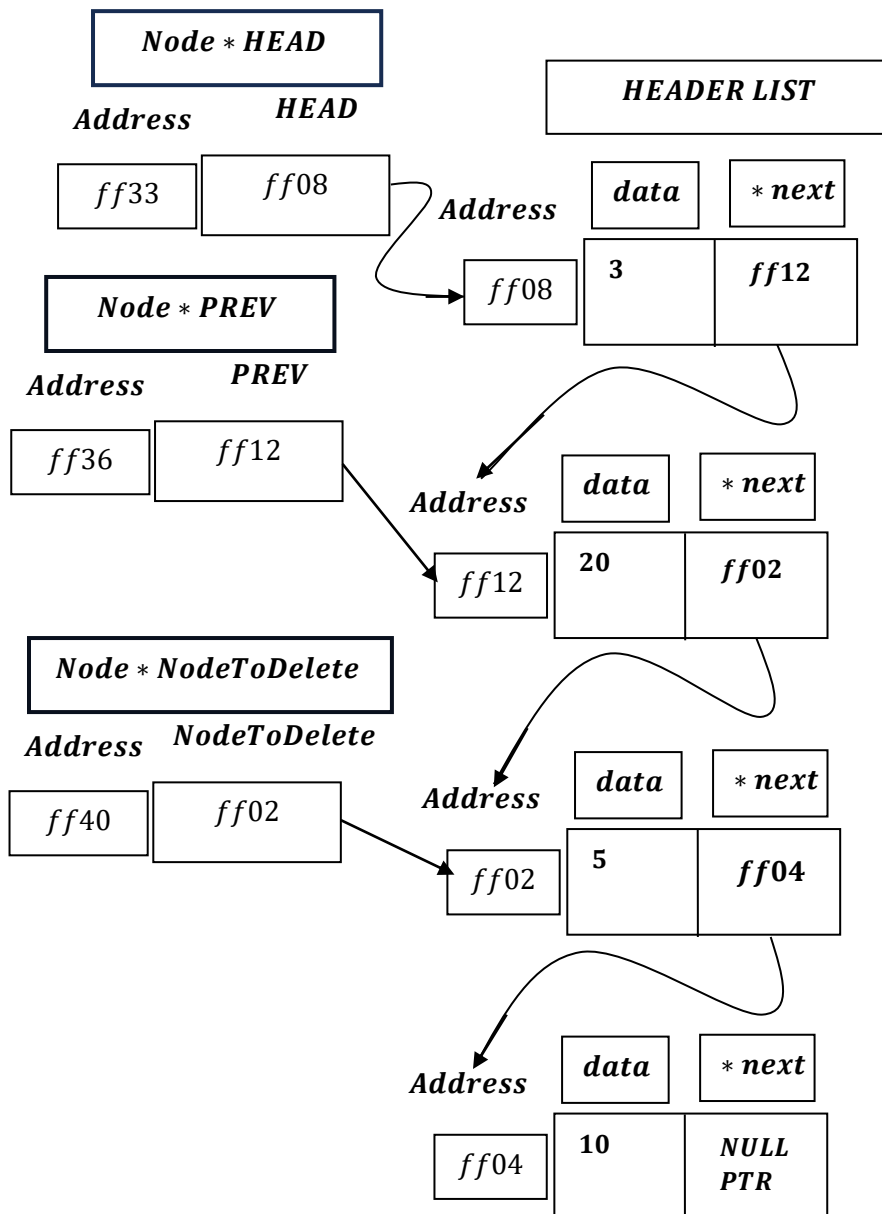


$i++ \Rightarrow i = i + 1 = 1 + 1 = 2.$

$i = 2 < \text{Pos}: 2$ [*False*], hence , exit from loop.

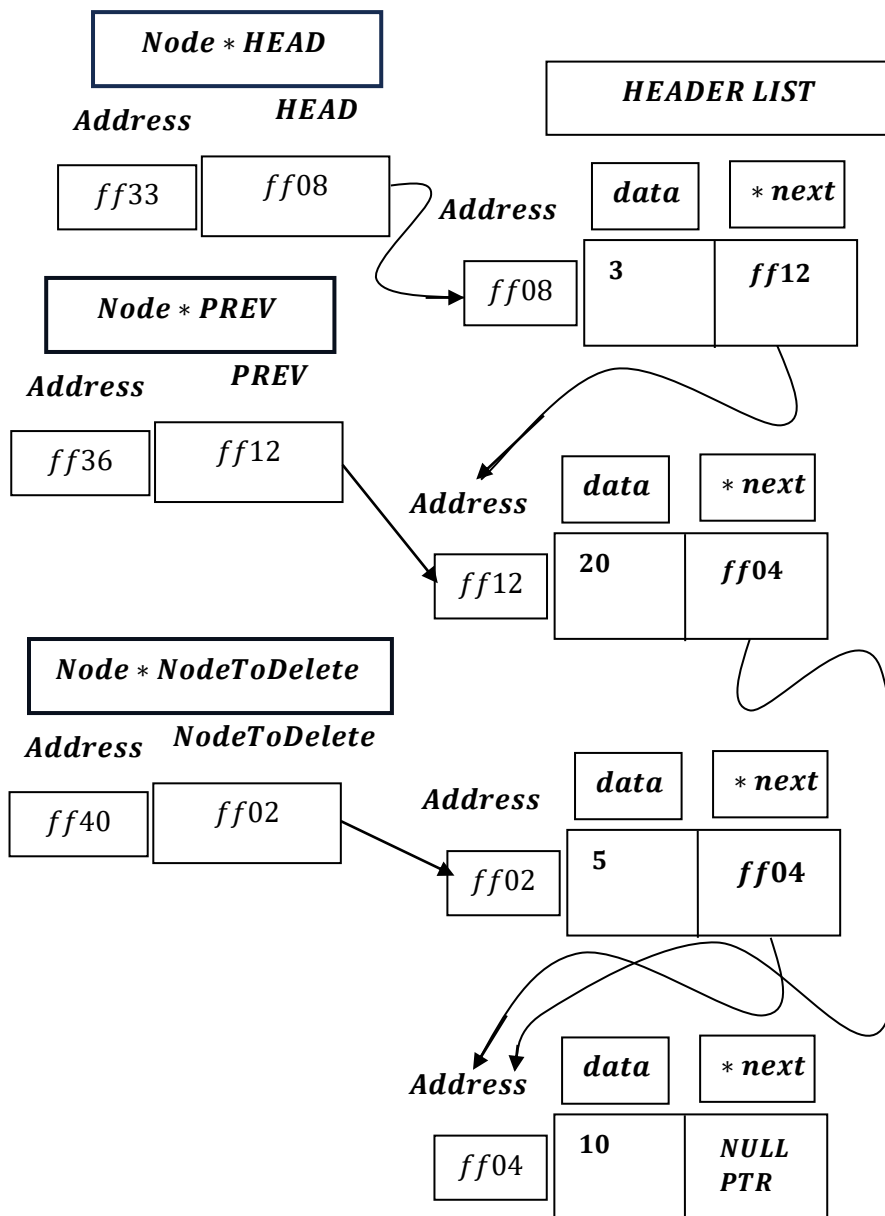
```
Node *nodeToDelete = temp->next;
```

$\text{NodeToDelete} = \text{PREV} \rightarrow \text{NEXT} = \text{ff02}$.



```
prev->next = nodeToDelete->next;
```

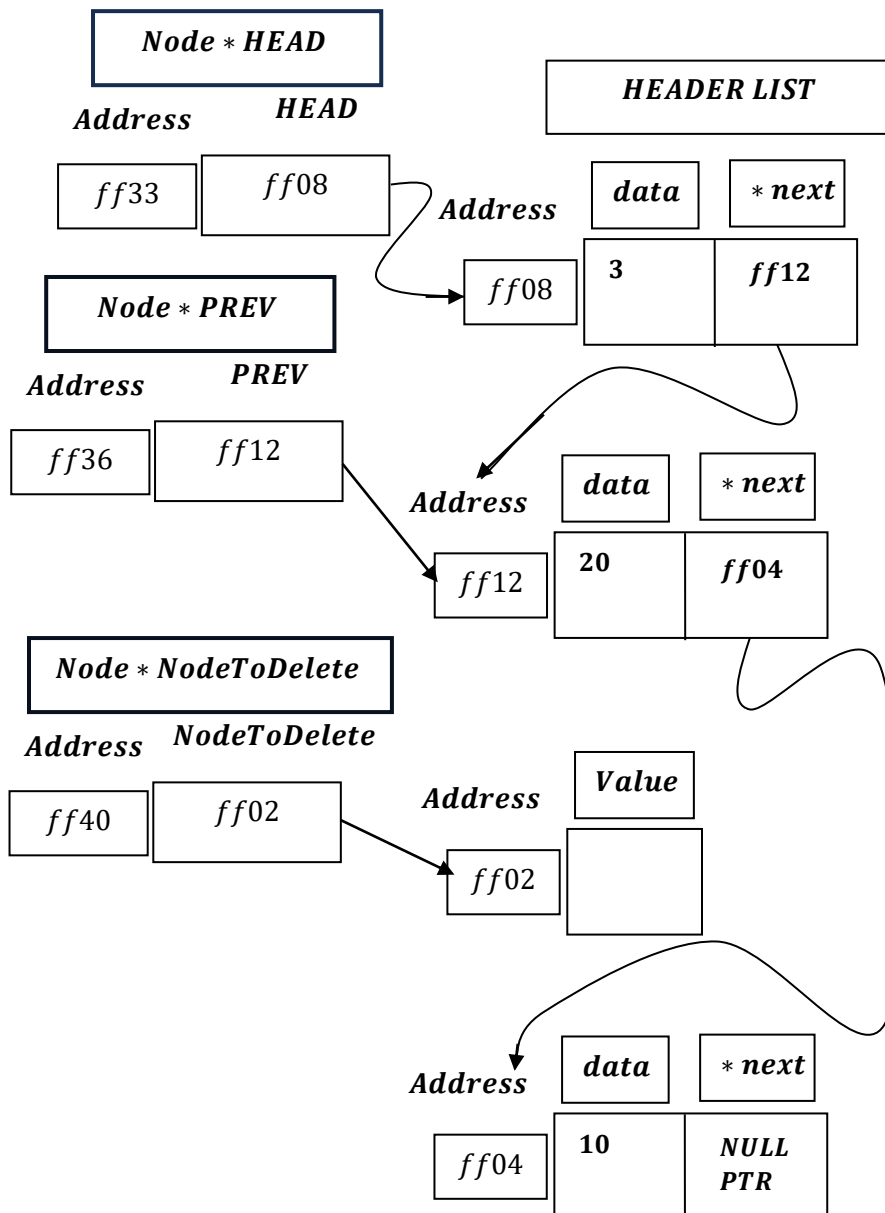
$\text{PREV} \rightarrow \text{NEXT} = \text{NodeToDelete} \rightarrow \text{NEXT} = \text{ff04}$.



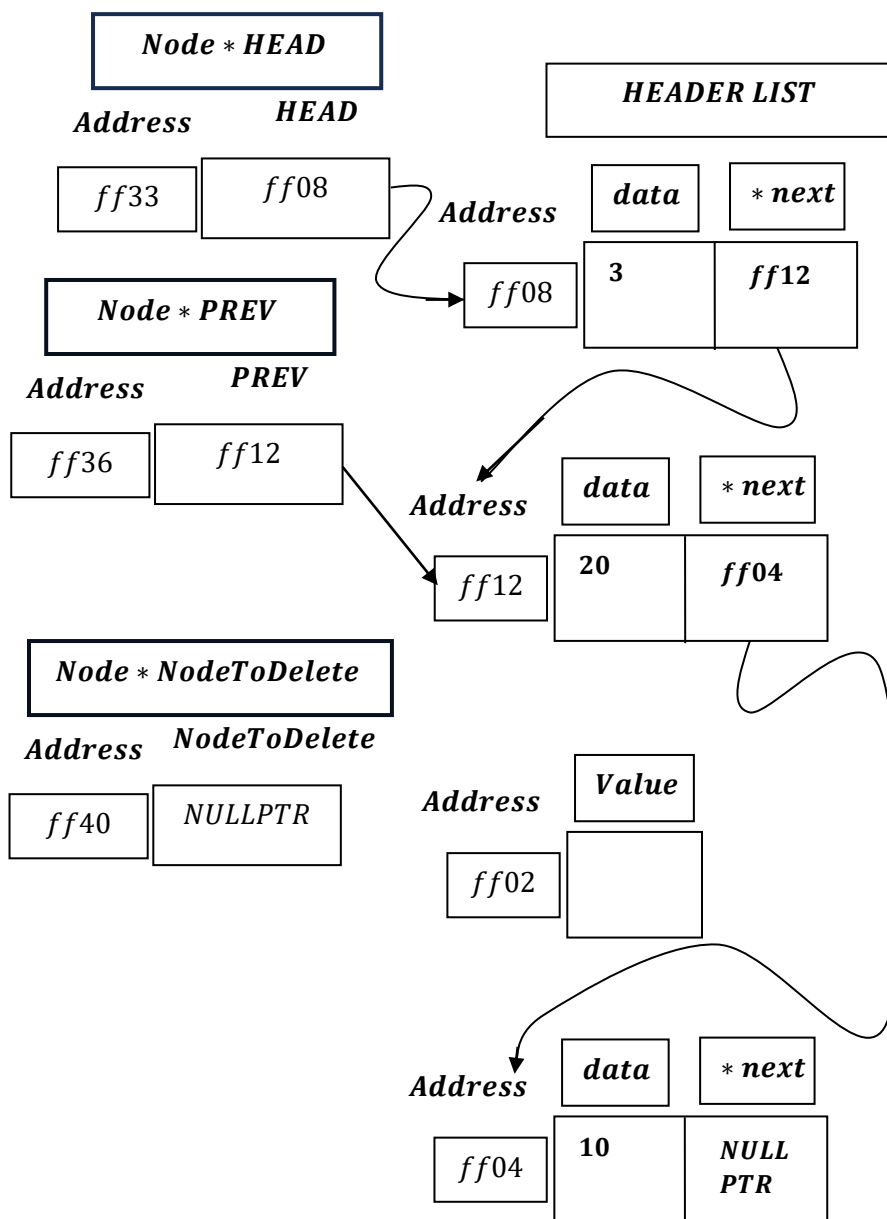
```
cout << "Deleted " << nodeToDelete->data << " from
position " << pos << "." << endl;
```

Output $\Rightarrow$ Print: Deleted `5` from position `2` .


```
free(nodeToDelete);
```

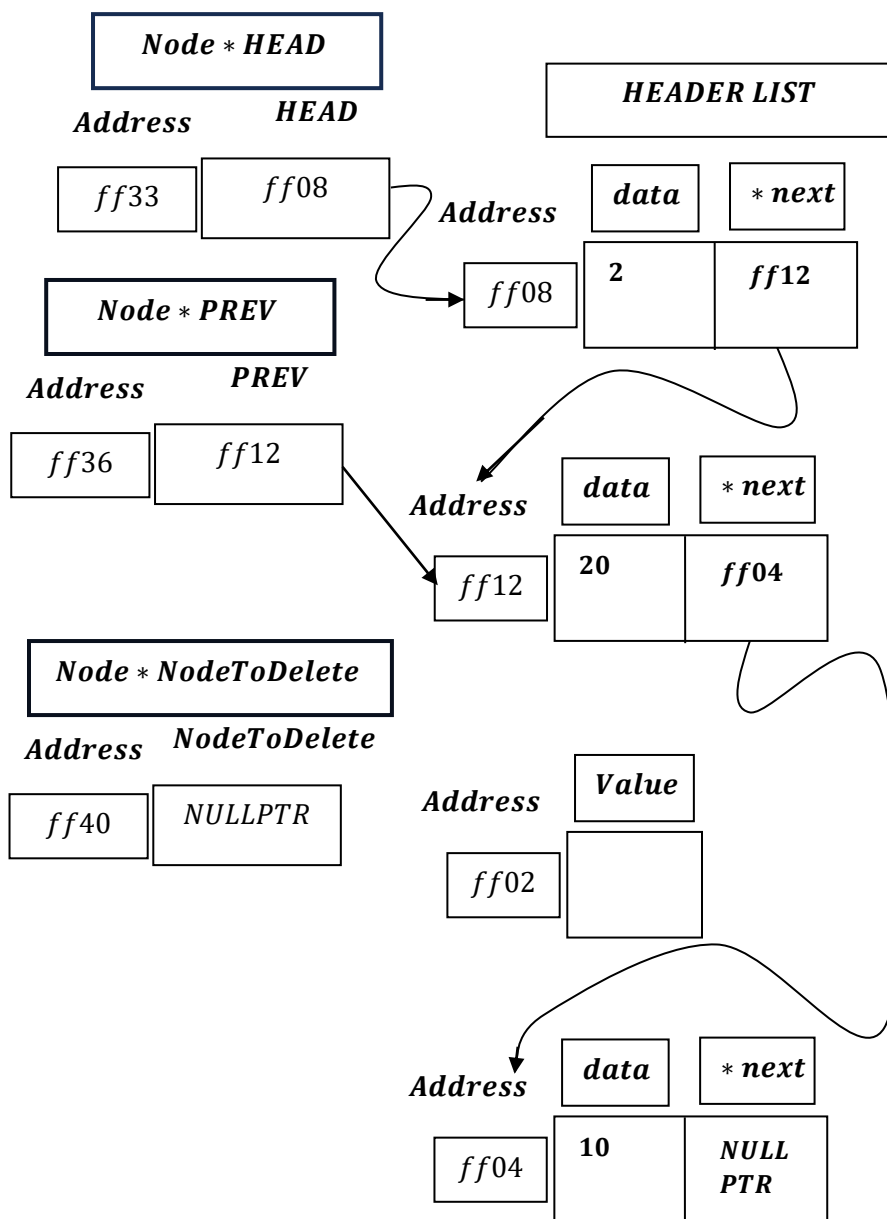


```
nodeToDelete = nullptr;
```



```
head->data--;
```

$Head \rightarrow Data -- \Rightarrow Head \rightarrow Data = Head \rightarrow Data - 1 = 3 - 1 = 2.$



And we get our final list:

